AllSpeak

A Language-Agnostic Runtime for Computational Literacy in Multilingual Communities

Working Paper — First Draft Framework

Abstract

The tools of software development are changing faster than the assumptions that underpin them. For decades, programming literacy meant the ability to write code; today, AI systems generate functional code from natural language descriptions with increasing reliability. This shift is already observable in professional practice — in changing patterns of developer activity, in the declining attendance at traditional coding meetups, in the growing proportion of codebases where no single human understands the whole.

What is less discussed is what this shift leaves behind. If the ability to write code is no longer the primary requirement, the ability to read and reason about code becomes more important, not less — and yet it is the skill least supported by current tools and education. A codebase that no human can meaningfully inspect is not a managed system; it is an autonomous one, operating on trust. In high-stakes domains — automation, finance, health, infrastructure — that is a governance problem as much as a technical one.

AllSpeak is an open-source scripting language and runtime designed for a world in which reading code matters more than writing it. Programs expressed in AllSpeak are intentionally simple, constrained, and readable — not because the problems they address are trivial, but because legibility is a design goal. Crucially, AllSpeak operates in any human language: the code itself, not merely the interface around it, is written in the learner's or reviewer's own tongue.

This paper describes AllSpeak's current state, the methodology for extending it to new languages, and the case for treating multilingual code legibility as a digital inclusion priority. It does not argue that traditional programming languages will disappear, nor that AI coding tools are unwelcome. It argues that as AI-generated code becomes ubiquitous, the humans who need to understand it will be a different population from those who once wrote it — less technically trained, more linguistically diverse — and that the tools available to them should reflect that reality.

1. Introduction

The global expansion of coding education over the past two decades has been built on a single unstated assumption: that the people learning to code will be the people writing it. Tools, curricula, and communities have been designed accordingly — for the authorship of code, in programming languages that are structurally rooted in English.

That assumption is now being tested. AI systems capable of generating functional code from natural language descriptions are in widespread professional use. The question of who writes code, and whether that skill remains the primary measure of computational literacy, is open in a way it was not five years ago.

AllSpeak is an open-source scripting language designed for a different assumption: that the more important skill, going forward, is the ability to read, understand, and meaningfully engage with code — and that this skill should be available to people regardless of their technical background or native language.

This paper describes AllSpeak's architecture, its current implementation in four languages, and the methodology developed for extending it to new linguistic communities. It also addresses directly the question that any honest treatment must confront: in an era of AI-generated code, what is the continuing value of a human-readable coding layer?

---

2. The Shifting Role of Code Literacy

2.1 What is already changing

The signals are not predictions — they are present-tense observations. Tools like GitHub Copilot have moved from novelty to standard workflow component in professional development environments within a few years of release. The proportion of code written with AI assistance is growing measurably. No-code and low-code platforms have expanded the population of people deploying functional software without writing conventional code. 'Prompt engineering' has emerged as a recognised professional skill.

These trends do not represent the end of programming. They represent a shift in where human skill and attention is applied in the development process. The practical question is what follows from that shift.

2.2 The handwriting parallel

An analogy is offered here not as a prediction but as a frame for thinking. When mechanical text reproduction became widely available — the typewriter, then the word processor — the teaching and practice of handwriting changed fundamentally. The ability to read handwritten text did not disappear; the ability to produce it fluently and elegantly largely did, because it was no longer the primary medium of written expression and was no longer systematically taught.

Whether code authorship follows a similar trajectory is a question reasonable people can disagree on. What is harder to dispute is that the population actively practising code writing is likely to shrink relative to the population that needs to engage with code in some way — reading it, questioning it, modifying it, deciding whether to trust it.

> **[NOTE TO AUTHOR:** *Add observational data here if available — meetup attendance trends, survey data on AI tool adoption among developers, any published studies on changing skill profiles.***]**

2.3 Reading versus writing

Whatever one believes about the future of code authorship, the need to read and reason about code is not diminishing. The volume of code requiring inspection, validation, and maintenance

is increasing. The population capable of performing that inspection is not growing proportionally, and the tools available to support it have not kept pace with the tools available for generating code.

This is a present-tense problem, not a future one. The gap between the rate of code generation and the rate of meaningful human review is already observable in professional contexts. It is likely to widen.

2.4 The closed loop problem

A system in which code is generated, deployed, and operated without meaningful human inspection is, in systems terms, an open loop: functional until it fails, with no internal mechanism for course correction short of that failure. In low-stakes contexts — generating a form letter, producing a data summary — this may be acceptable. In domains where the consequences of error are significant — home automation, financial calculation, medical scheduling, infrastructure control — it is a governance problem.

Constrained, legible runtimes are one class of response to this problem. They do not replace general-purpose languages; they occupy a different space — one where human comprehension of what the code does is a design requirement, not an afterthought. AllSpeak is designed for exactly this space.

2.5 The language dimension

If code literacy — reading, not writing — is the relevant skill for an expanding population, the language in which code is expressed becomes newly important. The population that needs to inspect and reason about AI-generated programs is not primarily composed of English-speaking developers. It includes teachers, community organisers, local administrators, and householders in communities where English is a second or third language.

Presenting code to these readers in English is not a neutral choice. It is a barrier. AllSpeak addresses this at the syntactic level: the code itself is in the reader's language, not merely the documentation around it.

---

3. Architecture and Approach

> **[NOTE TO AUTHOR:** *Technical section — to be developed in detail. The following is a structural placeholder.***]**

3.1 The opcode model

AllSpeak is built around approximately 100 canonical ALL_CAPS opcodes — the fixed, language-independent instruction set of the runtime. Each language pack maps these opcodes to keywords, patterns, and grammatical structures appropriate to that language. A program written in Italian and a program written in German execute identically; the execution engine sees only opcodes.

3.2 The JS/JSON architecture

Language packs are implemented as a split between JavaScript (structural and behavioural logic) and JSON (vocabulary and pattern data). This separation makes community contribution tractable: a translator working on a new language pack can contribute vocabulary and pattern data without engaging with the underlying JavaScript implementation.

3.3 The Weblate integration

Community translation and validation is managed through Weblate, hosted at translate.codeberg.org. This provides a structured workflow for native-speaker review, version control for language pack changes, and an audit trail for translation decisions.

---

4. Current Implementation

AllSpeak is currently operational in four languages: English (the reference implementation), French, German, and Italian. The three non-English implementations are functional — programs can be written and executed — but have not yet undergone formal linguistic validation by native-speaker educators.

This distinction matters. Functional translation means the opcodes map correctly and programs execute as expected. Validated translation means a native speaker with relevant educational experience has confirmed that the vocabulary choices are natural, unambiguous, and appropriate for the target audience. The validation step is a prerequisite for any serious claim of community suitability.

> **[NOTE TO AUTHOR:** *Describe validation methodology and timeline here once established. What does validation consist of? Who conducts it? What are the acceptance criteria?***]**

---

5. The Multilingual Challenge

5.1 Beyond vocabulary

Translating a programming language is not the same as translating a document. Vocabulary is the least of it. The deeper challenges arise from structural differences between languages: grammatical gender (which affects how keywords relate to each other), verb conjugation, sentence-final versus sentence-medial constructions, and the fundamental question of script.

5.2 Non-Latin scripts: Bulgarian as a case study

The inclusion of Bulgarian in AllSpeak's development roadmap is not incidental. Bulgarian, written in Cyrillic, represents a proof of concept for a genuinely language-agnostic system — one that does not treat the Latin alphabet as a default. The decisions made in implementing Bulgarian (including the question of whether to support Cyrillic natively, use transliteration, or offer both) establish a methodology applicable to Arabic, Devanagari, Chinese, and other non-Latin writing systems.

> **[NOTE TO AUTHOR:** *Expand with the specific decisions made on the Cyrillic/transliteration question and the reasoning behind them.***]**

---

6. Methodology for Language Onboarding

The following describes a replicable process for extending AllSpeak to a new language. It is intended to be executable by a small team — potentially a single motivated individual with AI assistance and access to native-speaker review.

6.1 AI-assisted initial translation

The reference English implementation provides the source material. An AI language model is used to produce an initial draft translation of the opcode vocabulary and pattern library. This draft is explicitly a starting point, not a finished product — AI translation of technical vocabulary into natural-sounding keywords in a target language requires human review.

6.2 Community review via Weblate

The draft translation is submitted to the Weblate instance for community review. Contributors with relevant language expertise can propose alternatives, flag unnatural choices, and discuss edge cases.

6.3 Educator validation

The reviewed translation is then assessed by at least one native-speaker educator with experience in the target community. Validation criteria include: naturalness of vocabulary choices; absence of ambiguity; suitability for the intended age range and educational context; and successful execution of the AllSpeak tutorial sequence (the Codex) in the target language.

6.4 The Codex as validation instrument

The AllSpeak Codex — a 20-step tutorial introducing the language incrementally — serves a dual purpose. For learners, it is an on-ramp. For validators, it is a test instrument: a language pack that cannot support a fluent Codex walkthrough is not ready for community use.

---

7. Educational and Community Applications

7.1 The Codex: computational literacy in days

A consistent finding in programming education is that the barrier to entry is not intellectual but temporal and psychological. Learners who cannot experience meaningful progress within hours or days rarely continue. The Codex is designed around this reality: twenty steps, each producing a visible result, each building on the last, with no assumed prior knowledge.

The four-step prompt series for building a simple graphical application extends this: a learner who has completed the Codex can, within a single session, produce something they can show to others. This is not trivial. The experience of producing something real is a significant determinant of whether learning continues.

7.2 Constrained automation: the RBR example

AllSpeak is not only an educational tool. It serves as the runtime for a working domestic heating management system, demonstrating that the constrained-runtime model has practical utility in real-world deployment contexts. A householder using such a system does not need to write code. But if the AI-generated heating schedule behaves unexpectedly, the ability to read a few lines of AllSpeak and identify the error is exactly the kind of human oversight that closed-loop governance requires.

> **[NOTE TO AUTHOR:** *Decide whether to name or anonymise the RBR project here. If named, add a brief technical description. If anonymised, refer to it as 'a working domestic automation deployment'.***]**

---

8. Open Questions and Limitations

8.1 Single-maintainer risk

AllSpeak's current development depends substantially on a single technical maintainer. This is a genuine vulnerability for any project seeking to position itself as community infrastructure. Mitigation strategies — contributor onboarding, documentation, institutional partnership — are recognised priorities.

8.2 Language pack governance

As the number of language packs grows, questions of governance arise. Who has authority to accept or reject changes to a language pack? How are disputes between contributors resolved? What happens to a language pack whose community contributors become inactive? These questions do not have settled answers and will require governance structures proportionate to the project's scale.

8.3 Sustainability

The long-term maintenance of a multilingual runtime is a resource commitment. Volunteer community contribution is valuable but not sufficient as a sole mechanism. The funding and institutional partnership questions addressed in this paper's conclusion are not incidental — they are existential for the project at the scale being described.

8.4 The AI dependency

The translation methodology described here relies substantially on AI language model capabilities. This is a practical efficiency, but it introduces a dependency that should be acknowledged: the quality of AI-assisted translation varies by language, and is generally poorer for lower-resource languages — precisely those where AllSpeak's inclusion argument is strongest. Human review is not optional; it is the quality control mechanism that makes AI-assisted translation viable.

---

9. Conclusion and Call for Collaboration

AllSpeak represents one response to a set of questions that the computing community is only beginning to take seriously: what does computational literacy mean when AI writes the code? Who needs to understand that code, and in what language? What tools exist to support them?

The project is at an early stage. Four languages are operational; three require validation. The methodology for language onboarding is established but not yet documented at the level required for independent replication. The governance and sustainability questions are identified but not resolved.

What this paper offers is a framework and an invitation. The framework is a coherent account of why multilingual code legibility matters and how it can be approached technically. The invitation is to linguists, educators, developers, and funding bodies who recognise the problem and have the capacity to help address it.

AllSpeak is open source. Its code, its language packs, and its methodology are available for inspection, contribution, and critique. That openness is not incidental — it is the project's most important property.

References

[NOTE TO AUTHOR: *References to be added. Likely to include: GitHub Copilot adoption studies; no-code/low-code market research; computational literacy education literature; UNESCO digital inclusion policy documents; Weblate and Codeberg project documentation; relevant prior work on localised programming languages (Scratch, etc.).***]**

Appendix: Technical Notes

[NOTE TO AUTHOR: *To be developed. Likely to include: complete opcode reference; language pack structure specification; Weblate workflow documentation; validation criteria checklist.***]**